MLDL Experiment 2

Aim :- Implement Linear and Logistic Regression on real-world datasets

1.Introduction

Regression analysis is a fundamental technique in machine learning used to predict continuous numerical outcomes based on one or more input variables. In real-world applications such as engineering, energy modeling, and demand forecasting, accurate prediction of continuous values is essential.

This experiment aims to implement and compare three regression techniques:

- Multiple Linear Regression (MLR)
- Ridge Regression
- Lasso Regression

The objective is to analyze their mathematical foundations, performance behavior, and the effect of regularization on prediction accuracy, multicollinearity handling, and feature selection.

By applying these algorithms to real-world datasets, the experiment evaluates their strengths, limitations, and suitability under different data conditions.

1. **Multiple Linear Regression (MLR)**

Multiple Linear Regression models the linear relationship between a dependent variable and multiple independent variables. It estimates coefficients by minimizing the sum of squared errors between predicted and actual values.

It serves as a baseline model and works well when:

- The relationship between variables is linear
- Multicollinearity is minimal
- The dataset follows regression assumptions

However, it may suffer from overfitting or instability when predictors are highly correlated.

Mathematical equation :

Model Equation

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$$

Where:

- y = Dependent (target) variable
- $x_1, x_2, \ldots, x_p$ = Independent variables
- β_0 = Intercept
- $\beta_1, \ldots, \beta_p$ = Regression coefficients
- ε = Error term

2. Ridge Regression

Ridge Regression is a regularized version of linear regression that adds an L2 penalty term to the loss function. This penalty shrinks coefficient values toward zero but does not eliminate them completely.

Ridge is particularly useful when:

- Multicollinearity exists
- The dataset contains highly correlated predictors
- Model stability and variance reduction are required

It improves generalization by reducing model complexity.

Mathematical equation :

Ridge Regression introduces an **L2 regularization penalty**.

Objective Function

$$\min_{\beta} \left[\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \right]$$

Where:

- $\lambda \geq 0$ = Regularization parameter
- $\sum \beta_j^2$ = L2 penalty term

Matrix Solution

$$\hat{\beta}_{ridge} = (X^T X + \lambda I)^{-1} X^T y$$

3. Lasso Regression

Lasso Regression introduces an L1 penalty to the loss function. Unlike Ridge, Lasso can shrink some coefficients exactly to zero, effectively performing feature selection.

Lasso is useful when:

- The dataset contains irrelevant features
- Model interpretability is important
- Automatic feature selection is required

It produces simpler and more interpretable models compared to standard regression.

Lasso introduces an **L1 regularization penalty**.

Objective Function

$$\min_{\beta} \left[\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \right]$$

Where:

- $\lambda \geq 0$ = Regularization parameter
- $\sum |\beta_j|$ = L1 penalty term

2. Dataset Source

The datasets used in this experiment are obtained from Kaggle, a widely recognized platform for real-world machine learning datasets.

Concrete Compressive Strength Dataset

3. Dataset Description

The Concrete Compressive Strength dataset contains information about the ingredients used in concrete mixtures and their corresponding compressive strength.

Dataset Size

- Total Instances: ~1030
- Number of Features: 8 independent variables
- Target Variable: Concrete Compressive Strength
- All features are numerical

Features Description

Feature	Description
Cement	Amount of cement (kg/m^3)
Blast Furnace Slag	Amount of slag (kg/m^3)
Fly Ash	Amount of fly ash (kg/m^3)
Water	Water content (kg/m^3)
Superplasticizer	Chemical admixture (kg/m^3)
Coarse Aggregate	Coarse aggregate (kg/m^3)
Fine Aggregate	Fine aggregate (kg/m^3)
Age	Age of concrete (days)

Target Variable:

- Concrete Compressive Strength (MPa)

4. Methodology / Workflow

The experiment follows a systematic machine learning pipeline to implement and compare Multiple Linear Regression, Ridge Regression, and Lasso Regression.

Step 1: Data Collection

- Imported datasets from Kaggle.
- Loaded data into the Python environment using Pandas.

Step 2: Data Preprocessing

- Checked for missing values and handled them appropriately.
- Performed encoding of categorical variables (if present).
- Applied feature scaling (Standardization) for Ridge and Lasso regression.
- Separated independent variables (X) and target variable (y).

Step 3: Train-Test Split

- Split the dataset into **training (80%)** and **testing (20%)** sets.
- Ensured proper random state for reproducibility.

Step 4: Model Implementation

- Implemented:
    - Multiple Linear Regression
    - Ridge Regression
    - Lasso Regression
- Trained each model on the training dataset.

Step 5: Model Evaluation

- Predicted values on the test dataset.
- Evaluated models using:
    - Mean Squared Error (MSE)
    - Root Mean Squared Error (RMSE)
    - Mean Absolute Error (MAE)
    - R^2 Score

Step 6: Hyperparameter Tuning

- Tuned regularization parameter (λ / alpha) for Ridge and Lasso using cross-validation.
- Selected optimal value based on best validation performance.

Step 7: Performance Comparison

- Compared all three models based on evaluation metrics.
- Analyzed coefficient behavior and effect of regularization.
- Interpreted which model performs best for each dataset.

5.Code Implementation

```
# ================================
# STEP 1: Upload Dataset
# ================================
from google.colab import files
uploaded = files.upload()  # Upload Concrete_Data.xls

# ================================
# STEP 2: Import Libraries
# ================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

from sklearn.model_selection import train_test_split, GridSearchCV,
cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
# ================================
# STEP 3: Load Dataset
# ================================
file_name = list(uploaded.keys())[0]
data = pd.read_excel(file_name)

print("\nDataset Loaded Successfully!")
print("Dataset Shape:", data.shape)
print("\nFirst 5 Rows:")
print(data.head())

# Rename target column (if needed)
data.rename(columns={
   'Concrete compressive strength(MPa, megapascals) ': 'Strength',
   'Concrete compressive strength(MPa, megapascals)': 'Strength'
}, inplace=True)

# ================================
# STEP 4: Data Preprocessing
# ================================
print("\nMissing Values:\n", data.isnull().sum())

X = data.drop("Strength", axis=1)
y = data["Strength"]

# ================================
# STEP 5: Correlation Heatmap
# ================================
plt.figure(figsize=(10,8))
sns.heatmap(data.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

```
# ================================
# STEP 6: Train-Test Split
# ================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ================================
# STEP 7: Feature Scaling
# ================================
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ================================
# STEP 8: Multiple Linear Regression
# ================================
mlr = LinearRegression()
mlr.fit(X_train_scaled, y_train)
y_pred_mlr = mlr.predict(X_test_scaled)

# ================================
# STEP 9: Ridge Regression + Tuning
# ================================
ridge_params = {"alpha": np.logspace(-3, 3, 50)}
ridge_grid = GridSearchCV(Ridge(), ridge_params, cv=5)
ridge_grid.fit(X_train_scaled, y_train)
best_ridge = ridge_grid.best_estimator_
y_pred_ridge = best_ridge.predict(X_test_scaled)

# ================================
# STEP 10: Lasso Regression + Tuning
# ================================
lasso_params = {"alpha": np.logspace(-3, 3, 50)}
```

```
lasso_grid = GridSearchCV(Lasso(max_iter=10000), lasso_params, cv=5)
lasso_grid.fit(X_train_scaled, y_train)
best_lasso = lasso_grid.best_estimator_
y_pred_lasso = best_lasso.predict(X_test_scaled)

# ================================
# STEP 11: Evaluation Function
# ================================
def evaluate(y_true, y_pred):
    return [
        r2_score(y_true, y_pred),
        np.sqrt(mean_squared_error(y_true, y_pred)),
        mean_absolute_error(y_true, y_pred)
    ]

# ================================
# STEP 12: Performance Comparison
# ================================
results = pd.DataFrame({
    "Model": ["MLR", "Ridge", "Lasso"],
    "R2 Score": [
        evaluate(y_test, y_pred_mlr)[0],
        evaluate(y_test, y_pred_ridge)[0],
        evaluate(y_test, y_pred_lasso)[0]
    ],
    "RMSE": [
        evaluate(y_test, y_pred_mlr)[1],
        evaluate(y_test, y_pred_ridge)[1],
        evaluate(y_test, y_pred_lasso)[1]
    ],
    "MAE": [
        evaluate(y_test, y_pred_mlr)[2],
        evaluate(y_test, y_pred_ridge)[2],
        evaluate(y_test, y_pred_lasso)[2]
```

```
    ]
})

print("\nModel Performance Comparison:")
print(results)

# ================================
# STEP 13: Cross Validation Scores
# ================================
print("\nCross Validation R2 Scores:")
for name, model in {
    "MLR": LinearRegression(),
    "Ridge": best_ridge,
    "Lasso": best_lasso
}.items():
    scores = cross_val_score(model, X_train_scaled, y_train, cv=5, scoring="r2")
    print(f"{name}: Mean R2 = {scores.mean()}")

# ================================
# STEP 14: Coefficient Comparison
# ================================
coef_df = pd.DataFrame({
    "Feature": X.columns,
    "MLR": mlr.coef_,
    "Ridge": best_ridge.coef_,
    "Lasso": best_lasso.coef_
})

print("\nCoefficient Comparison:")
print(coef_df)

# ================================
# STEP 15: Actual vs Predicted Plot
# ================================
```

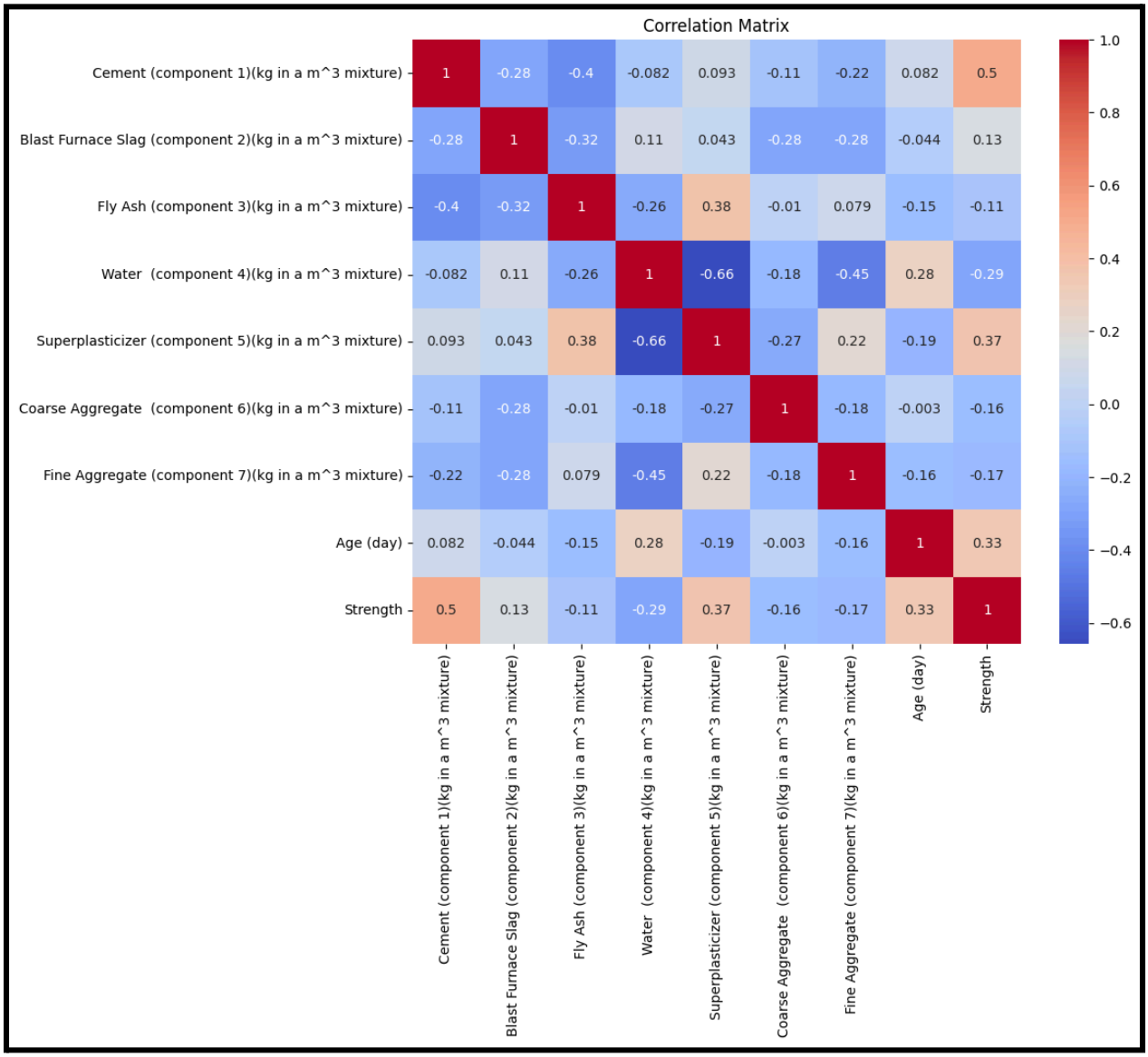
```
plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred_ridge)
plt.xlabel("Actual Strength")
plt.ylabel("Predicted Strength")
plt.title("Actual vs Predicted (Best Ridge)")
plt.show()
```

6.Output:-

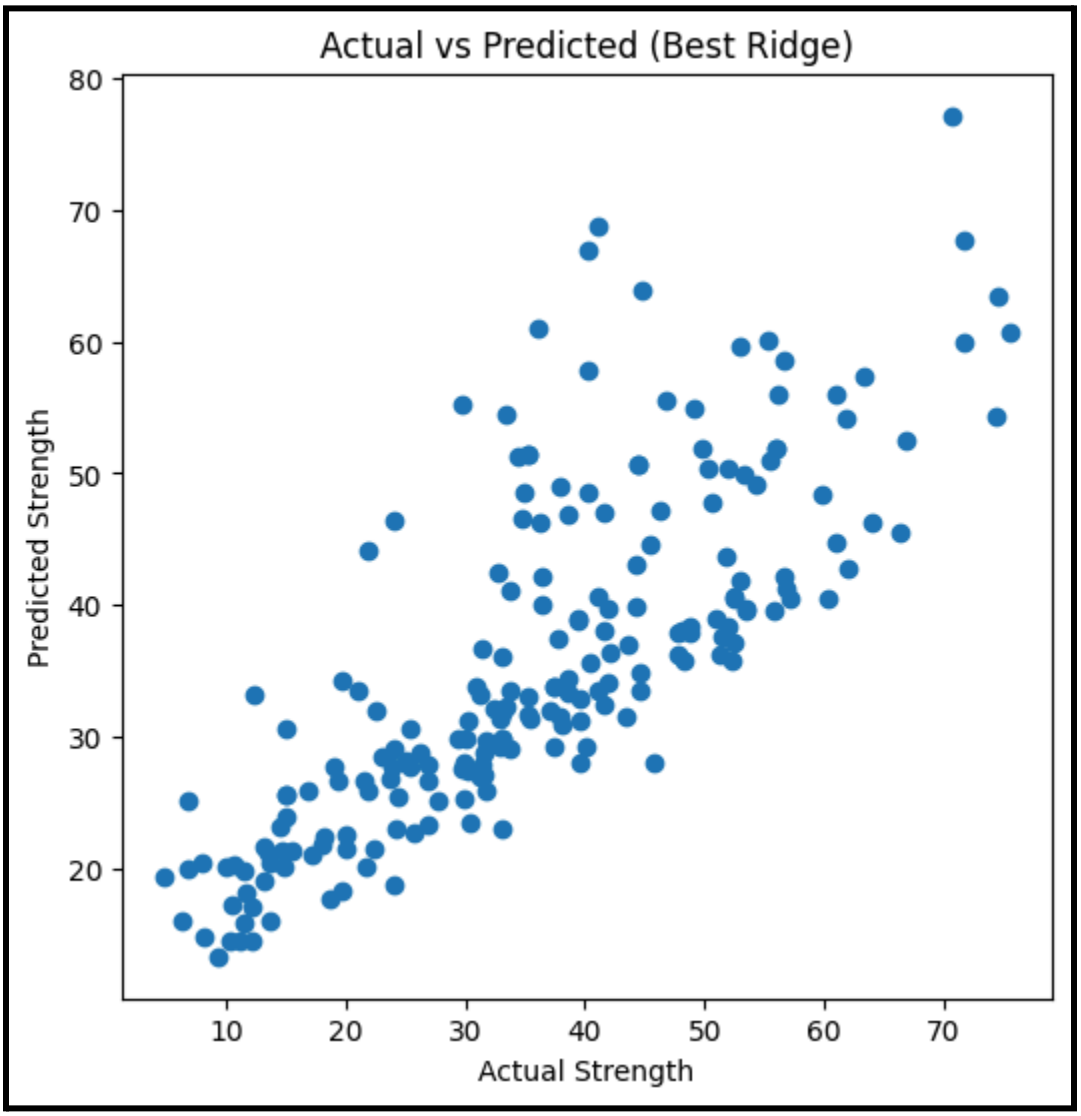
```
Choose Files  Concrete_Data.xls
Concrete_Data.xls(application/vnd.ms-excel) - 137216 bytes, last modified: 2/22/2026 - 100% done
Saving Concrete_Data.xls to Concrete_Data (1).xls

Dataset Loaded Successfully!
Dataset Shape: (1030, 9)

First 5 Rows:
   Cement (component 1)(kg in a m^3 mixture)  \
0                                      540.0
1                                      540.0
2                                      332.5
3                                      332.5
4                                      198.6

   Blast Furnace Slag (component 2)(kg in a m^3 mixture)  \
0                                                    0.0
1                                                    0.0
2                                                  142.5
3                                                  142.5
4                                                  132.4
```

```
•••     Fly Ash (component 3)(kg in a m^3 mixture)  \
     0                                          0.0
     1                                          0.0
     2                                          0.0
     3                                          0.0
     4                                          0.0

        Water  (component 4)(kg in a m^3 mixture)  \
     0                                       162.0
     1                                       162.0
     2                                       228.0
     3                                       228.0
     4                                       192.0

        Superplasticizer (component 5)(kg in a m^3 mixture)  \
     0                                                   2.5
     1                                                   2.5
     2                                                   0.0
     3                                                   0.0
     4                                                   0.0

        Coarse Aggregate  (component 6)(kg in a m^3 mixture)  \
     0                                                1040.0
     1                                                1055.0
     2                                                 932.0
     3                                                 932.0
     4                                                 978.4
```

```
...     Fine Aggregate (component 7)(kg in a m^3 mixture)  Age (day)  \
    0                                              676.0         28
    1                                              676.0         28
    2                                              594.0        270
    3                                              594.0        365
    4                                              825.5        360

       Concrete compressive strength(MPa, megapascals)
    0                                        79.986111
    1                                        61.887366
    2                                        40.269535
    3                                        41.052780
    4                                        44.296075

    Missing Values:
     Cement (component 1)(kg in a m^3 mixture)               0
    Blast Furnace Slag (component 2)(kg in a m^3 mixture)    0
    Fly Ash (component 3)(kg in a m^3 mixture)               0
    Water  (component 4)(kg in a m^3 mixture)                0
    Superplasticizer (component 5)(kg in a m^3 mixture)      0
    Coarse Aggregate  (component 6)(kg in a m^3 mixture)     0
    Fine Aggregate (component 7)(kg in a m^3 mixture)        0
    Age (day)                                                0
    Strength                                                 0
    dtype: int64
```



```
Model Performance Comparison:
   Model  R2 Score      RMSE       MAE
0    MLR  0.627542  9.796708  7.745393
1  Ridge  0.627499  9.797262  7.761991
2  Lasso  0.627551  9.796582  7.748174

Cross Validation R2 Scores:
MLR: Mean R2 = 0.5945342206332564
Ridge: Mean R2 = 0.5947816755240883
Lasso: Mean R2 = 0.5945409108005557

Coefficient Comparison:
                                          Feature        MLR      Ridge  \
0          Cement (component 1)(kg in a m^3 mixture)  12.786504  12.212343
1  Blast Furnace Slag (component 2)(kg in a m^3 m...   9.432883   8.877847
2         Fly Ash (component 3)(kg in a m^3 mixture)   5.255609   4.764208
3          Water  (component 4)(kg in a m^3 mixture)  -2.892085  -3.258527
4  Superplasticizer (component 5)(kg in a m^3 mix...   1.841103   1.848326
5  Coarse Aggregate  (component 6)(kg in a m^3 mi...   1.400255   1.019228
6  Fine Aggregate (component 7)(kg in a m^3 mixture)   1.947380   1.455130
7                                        Age (day)   7.037787   6.988182

       Lasso
0  12.678571
1   9.326307
2   5.163134
3  -2.970094
4   1.832680
5   1.318671
6   1.847011
7   7.027935
```



7.Conclusion

In this experiment, Multiple Linear Regression, Ridge Regression, and Lasso Regression were implemented on the Concrete Compressive Strength dataset to predict the compressive strength of concrete based on its components.

Multiple Linear Regression served as the baseline model and provided good predictive performance under the assumption of linear relationships between features and the target variable. However, it showed sensitivity to multicollinearity among predictors.

Ridge Regression improved model stability by applying L2 regularization, which reduced coefficient magnitudes and minimized variance caused by correlated features. It demonstrated better generalization performance compared to standard linear regression.

Lasso Regression applied L1 regularization and performed automatic feature selection by shrinking some coefficients to zero. This resulted in a simpler and more interpretable model while maintaining competitive predictive performance.

Hyperparameter tuning using cross-validation further enhanced the performance of Ridge and Lasso models by identifying optimal regularization parameters.

Overall, the experiment demonstrates that:

- Regularization improves model generalization.
- Ridge Regression is effective when multicollinearity is present.
- Lasso Regression is useful for feature selection and model simplification.
- Multiple Linear Regression works well as a baseline when assumptions are satisfied.

Thus, regularized regression techniques provide more robust and reliable models for real-world prediction problems compared to traditional linear regression alone.